

程序报告

学号：2312331

姓名：王一诺

一、问题重述

（简单描述对问题的理解，从问题中抓住主干，必填）

1.1 实验背景

今年一场席卷全球的新型冠状病毒给人们带来了沉重的生命财产的损失。
有效防御这种传染病毒的方法就是积极佩戴口罩。
我国对此也采取了严肃的措施，在公共场合要求人们必须佩戴口罩。
在本次实验中，我们要建立一个目标检测的模型，可以识别图中的人是否佩戴了口罩。

1.2 实验要求

- 1) 建立深度学习模型，检测出图中的人是否佩戴了口罩，并将其尽可能调整到最佳状态。
- 2) 学习经典的模型 MTCNN 和 MobileNet 的结构。
- 3) 学习训练时的方法。

1.3 实验环境

可以使用基于 Python 的 OpenCV、PIL 库进行图像相关处理，使用 Numpy 库进行相关数值运算，使用 Pytorch 等深度学习框架训练模型等。

1.4 注意事项

Python 与 Python Package 的使用方式，可在右侧 API 文档 中查阅。
当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

1.5 参考资料

论文
Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks:
https://kpzhang93.github.io/MTCNN_face_detection_alignment/
OpenCV :
https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_tutorials.html
PIL: <https://pillow.readthedocs.io/en/stable/>
Numpy: <https://www.numpy.org/>
Scikit-learn: <https://scikit-learn.org/>
PyTorch: <https://pytorch.org/>

二、设计思想

（所采用的方法，有无对方法加以改进，该方法有哪些优化方向（参数调整，框架调整，或者指出方法的局限性和常见问题），伪代码，理论结果验证等... 思考题，非必填）

针对目标检测的任务，可以分为两个部分：目标识别和位置检测。
通常情况下，特征提取需要由特有的特征提取神经网络来完成，如 VGG、MobileNet、

ResNet 等,这些特征提取网络往往被称为 Backbone。而在 Backbone 后面接全连接层(FC)就可以执行分类任务。

但 FC 对目标的位置识别乏力。经过算法的发展,当前主要以特定的功能网络来代替 FC 的作用,如 Mask-Rcnn、SSD、YOLO 等。

我们选择充分使用已有的人脸检测的模型,再训练一个识别口罩的模型,从而提高训练的开支、增强模型的准确率。

项目文件中给出了较完整的 MobileNet 模型的搭建和训练过程,我们仿照其设计实现和训练 MobileNet 网络,并通过修改调试几个关键参数来降低损失值、提高正确率。

以下几个语句中包含了几个关键的参数:

```
# 3.创建模型和训练模型,训练模型时尽量将模型保存在 results 文件夹
epochs = 2
model = MobileNetV1(classes=2).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
# 学习率下降
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', factor=0.5, patience=2)
```

我们关注调节的就是 epochs (总训练轮数)、lr、factor 和 patience;

观察 loss 变化,它会随着 epochs 的增大而先下降后增大(过拟合的原因)。一般 loss 在 30-45 轮左右达到最低,得到最佳模型。我们先测试原有参数情况下的得分:

测试点	状态	时长	结果
在 5 张图片上测试模型	✓	5s	得分:55.0

分数很低,尝试增大 epochs 至 30:

测试点	状态	时长	结果
在 5 张图片上测试模型	✓	4s	得分:90.0

分数已经很高了,但没达到最好。尝试把 epochs 改为 40,并调试 factor 和 patience:

```
epochs = 40
model = MobileNetV1(classes=2).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
# 学习率下降
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', factor=0.3, patience=5)
```

几次修改,得分仍然是 90:

测试点	状态	时长	结果
在 5 张图片上测试模型	✓	5s	得分:90.0

那我们尝试修改 epochs,在 30-40 轮数中调试试探,最终发现在 epochs=33 时得到最佳得分:

```

epochs = 33
model = MobileNetV1(classes=2).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
# 学习率下降
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', factor=0.5, patience=2)

```

测试点	状态	时长	结果
在 5 张图片上测试模型		5s	得分:100.0

三、代码内容

（能体现解题思路的主要代码，有多个文件或模块可用多个"===="隔开，**必填**）

```

=====train.py=====

import warnings
# 忽视警告
warnings.filterwarnings('ignore')
import cv2
from PIL import Image
import numpy as np
import copy
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.datasets import ImageFolder
import torchvision.transforms as T
from torch.utils.data import DataLoader
from torch_py.Utills import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition
from torch_py.Utills import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition
from torch_py.FaceRec import Recognition
from PIL import Image
import cv2
# 1.加载数据并进行数据处理
def processing_data(data_path, height=224, width=224, batch_size=32,
test_split=0.1):
    transforms = T.Compose([
        T.Resize((height, width)),

```

```

        T.RandomHorizontalFlip(0.1), # 进行随机水平翻转
        T.RandomVerticalFlip(0.1), # 进行随机垂直翻转
        T.ToTensor(), # 转化为张量
        T.Normalize([0], [1]), # 归一化
    ])
    dataset = ImageFolder(data_path, transform=transforms)
    # 划分数据集
    train_size = int((1-test_split)*len(dataset))
    test_size = len(dataset) - train_size
    train_dataset, test_dataset = torch.utils.data.random_split(dataset, [train_size, test_size])
    # 创建一个 DataLoader 对象
    train_data_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    valid_data_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=True)
    return train_data_loader, valid_data_loader
data_path = './datasets/5f680a696ec9b83bb0037081-momodel/data/image'
# 2.如果有预训练模型，则加载预训练模型；如果没有则不需要加载
device = torch.device("cuda:0") if torch.cuda.is_available() else torch.device("cpu")
train_data_loader, valid_data_loader = processing_data(data_path=data_path, height=160,
width=160, batch_size=32)
# 3.创建模型和训练模型，训练模型时尽量将模型保存在 results 文件夹
epochs = 33
model = MobileNetV1(classes=2).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3) # 优化器
# 学习率下降
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'max', factor=0.5, patience=2)
# 损失函数
criterion = nn.CrossEntropyLoss()
best_loss = 1e9
best_model_weights = copy.deepcopy(model.state_dict())
loss_list = [] # 存储损失函数值
for epoch in range(epochs):
    model.train()
    for batch_idx, (x, y) in tqdm(enumerate(train_data_loader, 1)):
        x = x.to(device)
        y = y.to(device)
        pred_y = model(x)
        # print(pred_y.shape)
        # print(y.shape)
        loss = criterion(pred_y, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        if loss < best_loss:
            best_model_weights = copy.deepcopy(model.state_dict())

```

```

        best_loss = loss
        loss_list.append(loss)
        print('step:' + str(epoch + 1) + '/' + str(epochs) + ' || Total Loss: %.4f % (loss))
# 4.评估模型，将自己认为最佳模型保存在 result 文件夹，其余模型备份在项目中其它文件
夹，方便您加快测试通过。
torch.save(model.state_dict(), './results/temp2.pth')
print('Finish Training.')

=====main.py=====

import warnings
# 忽视警告
warnings.filterwarnings('ignore')

import cv2
from PIL import Image
import numpy as np
import copy
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.datasets import ImageFolder
import torchvision.transforms as T
from torch.utils.data import DataLoader
from torch_py.Utills import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition
from torch_py.Utills import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition
from torch_py.FaceRec import Recognition
from PIL import Image
import cv2
# ----- 请加载您最满意的模型 -----
# 加载模型(请加载你认为的最佳模型)
# 加载模型,加载请注意 model_path 是相对路径，与当前文件同级。
# 如果你的模型是在 results 文件夹下的 dnn.h5 模型，则 model_path = 'results/temp.pth'
model_path = 'results/temp.pth'
# -----
def predict(img):
    """

```

加载模型和模型预测

:param img: cv2.imread 图像

:return: 预测的图片中的总人数、其中佩戴口罩的人数

"""

----- 实现模型预测部分的代码 -----

将 cv2.imread 图像转化为 PIL.Image 图像，用来兼容测试输入的 cv2 读取的图像
(勿删!!!)

cv2.imread 读取图像的类型是 numpy.ndarray

PIL.Image.open 读取图像的类型是 PIL.JpegImagePlugin.JpegImageFile

if isinstance(img, np.ndarray):

 # 转化为 PIL.JpegImagePlugin.JpegImageFile 类型

 img = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

recognize = Recognition(model_path)

img, all_num, mask_num = recognize.mask_recognize(img)

return all_num, mask_num

四、实验结果

(实验结果, 必填)

tb_results	3 months ago
_README.md	3 months ago
temp.pth	an hour ago
temp2.pth	32 minutes ago

通过调参训练，得到最佳模型 temp.pth

系统测试

main.py

results

torch.py

接口测试

✓ 接口测试通过。

用例测试

测试点	状态	时长	结果
在 5 张图片上测试模型	✓	5s	得分:100.0

五、总结

(自评分析(是否达到目标预期, 可能改进的方向, 实现过程中遇到的困难, 从哪些方面可

以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

通过本次实验，学习到了训练和提升模型性能的一个关键手段——调参。这些参数都在哪些方面影响着训练效果？应该调整哪些参数？某个参数应该调整到什么成程度才好？这些都是我们需要考虑的问题，需要掌握模型训练过程、观察 loss 变化以明确调参方向，最终得到最佳效果模型。